Foundations of Computational Linguistics - Lecture 3 continued (13 Oct)

Session context

This cumulative deck combines material from the October 6 session with its October 13 continuation. The opening slides review the earlier material.

Lecture overview

The October 13 continuation begins with a recap of n-gram language models and the Markov assumption, then covers smoothing, intrinsic and extrinsic evaluation, and limitations caused by sparse data, short context windows, and missing semantic representations.

The continuation develops distributional representations: words are represented as vectors built from their contexts, and vector similarity captures semantic relationships. It then revisits logistic regression for classification and shows how feed-forward neural networks combine logistic-regression-like units across layers. Training is explained through prediction loss, mini-batches, stochastic gradient descent, and backpropagation. The final sections cover learned word embeddings, prediction-based objectives, Word2Vec's CBOW and Skip-gram approaches, negative sampling, nearest-neighbor similarity, analogies, and the ways corpus choice can shape or limit the resulting representations.

Slide 1: faculty of arts

This faculty title card is dated October 6, when Lecture 03 began. The October 13 recording continues the same lecture.

Slide 2: Tokenization

Computers encode text as a sequence of characters where words, spaces, and punctuation are treated identically. Because this uniform encoding obscures linguistic structure, tokenization is required to explicitly define and mark word boundaries within the string.

Slide 3: Tokenization

Tokenization addresses the fundamental question of defining a word as a consecutive sequence of characters separated by whitespace or punctuation. The slide highlights the specific challenge posed by apostrophes, illustrating how they complicate this definition in cases like "sms'en," "gsm'etje," and "doin'."

Slide 4: Tokenization

Tokenization addresses the fundamental question of defining a word as a consecutive sequence of characters separated by whitespace or punctuation. The slide highlights specific ambiguities in this definition, particularly regarding how to handle apostrophes (e.g., "sms'en", "doin'") and hyphens (e.g., "self-driving", "what-do-you-mean-look"). These examples illustrate the complexity of determining whether such characters act as separators or integral parts of a single token.

Slide 5: Tokenization

Tokenization involves defining a word as a consecutive sequence of characters separated by whitespace or punctuation, though this definition faces challenges with apostrophes (e.g., "doin'") and hyphens (e.g., "self-driving"). Complications arise in languages lacking separation markers, such as compound words in Dutch ("langeafstandsloper"), Chinese scripts without whitespace, and agglutinating languages like Turkish, Basque, and Inuit.

Slide 6: Tokenization

Tokenization involves determining what constitutes a "word" within a text. A key challenge arises with proper names, such as "San Francisco" and "New York," which consist of multiple tokens but represent single entities. This slide highlights the ambiguity in defining word boundaries when processing multi-word geographical names.

Slide 7: Tokenization

Tokenization addresses the fundamental challenge of defining what constitutes a "word" in computational linguistics. The slide highlights that this definition is complicated by proper names like "San Francisco" and fixed expressions such as "by and large," "piece of cake," and "spill the beans." These examples illustrate cases where standard word boundaries may not align with semantic units, requiring specific handling during the tokenization process.

Slide 8: Tokenization

Tokenization addresses the challenge of defining a "word," particularly for proper names like "San Francisco" and fixed expressions such as "by and large." Modern neural models circumvent this ambiguity by splitting text into frequency-based subword units, which can be smaller than whole words. A primary example of this approach is byte pair encoding, a topic that will be explored further later in the course.

Slide 9: Sentence detection

Sentence detection relies on punctuation, where characters like '?' and '!' are unambiguous markers of sentence boundaries. However, the dot ('.') is highly ambiguous because it appears in abbreviations such as "U.N." or "etc." within sentences. Consequently, context is essential to accurately determine whether a dot signifies the end of a sentence or an internal abbreviation.

Slide 10: Language models

Symbolic models provide a binary assessment of sentence well-formedness, whereas probabilistic language models estimate the likelihood of strings being valid, requiring all sentence probabilities to sum to one. While simple statistical approaches struggle with long-distance dependencies and lack meaning representation, more evolved neural models address these limitations by incorporating semantic information. Unlike interpretable symbolic rules, these complex models rely on parameter combinations that are difficult to tune individually but improve significantly with increased data volume.

Slide 11: Uses of language models

Language models improve speech recognition, OCR, handwriting recognition, machine translation, text generation, and context-sensitive spelling correction by selecting the most probable sentences. For instance, they distinguish "I ate a tomato" from phonetically similar but unlikely alternatives like "eye eight atom eight oh." Similarly, they identify correct readings or translations based on sentence likelihood and fix errors such as changing "Their is a problem wit this sentence" to its more probable form.

Slide 12: Completion prediction

Completion prediction refers to a language model's ability to predict the next word in a sentence, as illustrated by examples like "Please turn off your mobile ____" and "If he loses, he has only himself to ____." This concept underpins predictive text input systems, which guess what a user is typing and provide choices to complete the text.

Slide 13: Statistical language model

A statistical language model computes sentence probabilities by counting occurrences in a large corpus, though exact sentence matching is often impractical due to data sparsity. Instead, the model estimates conditional probabilities based on limited preceding word contexts to determine the likelihood of the next word. This simple counting approach effectively calculates the global probability of word sequences for applications like text generation and assessing sentence well-formedness.

Slide 14: Statistical language model

A statistical language model computes sentence probability by dividing the count of a specific sentence's occurrences in a corpus by the total number of sentences. For example, the probability of "the dog was running across the green field" is calculated as its frequency divided by 10^14, resulting in zero because the exact sentence does not appear in the data.

Slide 15: Statistical language model

A statistical language model computes the joint probability of a word sequence by decomposing it into a product of conditional probabilities. For the sentence "the dog was running across the green field," this involves multiplying terms such as p(the), p(dog|the), and subsequent words conditioned on all preceding context. This decomposition transforms the complex joint probability calculation into a series of manageable conditional probability estimations.

Slide 16: Statistical language model

Statistical language models compute the joint probability of a word sequence by decomposing it into a product of conditional probabilities. For the sentence "the dog was running across the green field," this involves multiplying terms like p(the) and p(dog|the), where each subsequent word depends on the preceding context. The model simplifies this computation by utilizing only a limited context rather than the entire history.

Slide 17: Statistical language model

Statistical language models compute the joint probability of a word sequence by decomposing it into a product of conditional probabilities. For the sentence "the dog was running across the green field," this involves multiplying terms like p(the) and p(dog|the), where each subsequent word depends on preceding ones. This approach relies on using only a limited context to estimate these probabilities efficiently.

Slide 18: Estimating probabilities

This slide presents formulas for estimating probabilities using word counts. The probability of a single word is the count of that word divided by the total token count, while conditional probabilities are calculated by dividing the count of a specific n-gram by the count of its preceding context. Examples include calculating p(dog|the) and p(green|across the) based on these frequency ratios.

Slide 19: N-gram models

N-gram models estimate the probability of a word given its prior context, where multiplying individual probabilities yields the total sentence probability. These models rely on the Markov assumption, predicting future states based only on a limited history of $n-1$ words rather than the entire sequence. While this simplification ignores broader context and may reduce accuracy in some cases, it remains a practical approach for language modeling.

Slide 20: Smoothing

Smoothing addresses data sparsity in probability estimation by preventing individual probabilities from becoming zero, which would otherwise nullify the entire sequence likelihood due to multiplication. Similar to Naive Bayes, techniques like add-one smoothing reassign probability mass to unseen events to ensure non-zero values. More advanced schemes such as Good-Turing, Back-off, and Kneser-Ney also exist to handle these estimation challenges effectively.

Slide 21: Evaluation

Language models are evaluated by training on one corpus portion and testing on held-out text, where good models assign high probabilities to unseen sentences. The standard metric, perplexity, represents the average number of words a model hesitates between at each position; lower values indicate better performance as they reflect higher certainty in word prediction. This approach constitutes intrinsic evaluation, which assesses the model's standalone ability to predict language rather than its extrinsic utility in applications like speech recognition or machine translation.

Slide 22: Inconveniences

The approach discussed is simple and effective for application to large text corpora. However, it remains an insufficient model of language due to its inability to handle long-distance dependencies and its lack of meaning representation. These limitations highlight the gap between statistical simplicity and true linguistic understanding.

Slide 23: Long-distance dependencies

Limited context models like n-grams fail to predict long-distance dependencies because the correct word depends on distant sentence elements rather than immediate predecessors. Syntactic examples show verb agreement relies on the initial subject noun ("man" vs. "men"), while semantic examples demonstrate that verb choice ("flies" vs. "talks") depends on the head noun's meaning. Consequently, more complex language models are required to capture these relationships spanning the entire sentence structure.

Slide 24: Lack of meaning

While discrete approaches in natural language processing work surprisingly well, they provide a poor representation of word meaning by treating words as atomic units. Although numerical probabilities for similar words like "apple" and "pear" are closer than those for unrelated words like "car," this method lacks an inherent understanding of semantic similarity. Consequently, word embeddings are introduced as a superior representation that allows networks to grasp how similar one word is to another without manual value assignment.

Slide 25: Lack of meaning

Although the discrete approach in natural language processing works surprisingly well, it provides a poor representation of word meaning by treating words as atomic units. Models like Naive Bayes calculate probabilities independently, failing to capture semantic similarities; for instance, knowing "fantastic" implies positivity does not help predict that the similar word "sensational" also indicates positivity. Consequently, these basic statistical models ignore the logical connections between semantically related terms.

Slide 26: Lack of meaning

The discrete approach is frequently used in natural language processing and performs surprisingly well, yet it remains a poor representation of word meaning. This method relies on atomic word representations where words are treated as isolated units without capturing semantic relationships, as illustrated by the binary vector examples for terms like "apple," "pear," "fantastic," and "sensational."

Slide 27: Lack of meaning

The discrete approach is frequently used in natural language processing and performs surprisingly well, yet it offers a very poor representation of word meaning. This method relies on atomic word representations where each word is treated as an independent unit, illustrated by probability vectors for terms like "apple" or "fantastic." Consequently, this representation fails to capture semantic relationships between words, such as the similarity between "apple" and "pear."

Slide 28: Vectors

A vector is an ordered list of numbers that places an item in a feature space. Word embeddings use this representation so words can be compared geometrically.

Slide 29: Vectors

This build continues the vector introduction. A word’s embedding is a point in a high-dimensional space; the dimensions are learned features rather than hand-labeled dictionary definitions.

Slide 30: Vectors

The vector pages lead into distributional semantics: words used in similar contexts can receive nearby representations, making geometric similarity useful for comparing words.

Slide 31: Introduction

Instead of discrete word representations, this slide introduces rich feature structures where words like "apple," "pear," and "car" are defined by properties such as being abstract, artifacts, or edible. This approach captures semantic similarity, as "apple" and "pear" share more features than either does with "car." However, because manually creating these structures is labor-intensive, the goal is to automatically extract these feature vectors from text rather than encoding them by hand.

Slide 32: Distributional similarity

Distributional similarity relies on the distributional hypothesis, which posits that a word's meaning can be inferred from its surrounding contexts. This approach models words as vectors within a matrix based on co-occurrence counts with context words, allowing mathematical measures to compute semantic similarity between them. Unlike bigram models focused on prediction, this method identifies which words are semantically similar by analyzing how frequently they appear in comparable environments.

Slide 33: Distributional similarity

Distributional similarity relies on the distributional hypothesis, which posits that a word's meaning can be inferred from its context. For instance, encountering an unknown term like "sooluceps" alongside descriptors such as "tasty," "sweet," and "freshly baked" allows one to deduce it refers to food. Consequently, semantic similarity is established by analyzing the surrounding words that define a term's usage.

Slide 34: Distributional similarity

Distributional similarity relies on the distributional hypothesis, which posits that a word's meaning can be inferred by examining its context. The slide illustrates this using the word "sooluceps" paired with various descriptors like "tasty," "sweet," "stale," and "freshly baked." By analyzing these surrounding contexts, one can determine the semantic properties of the target word without explicit definitions.

Slide 35: Matrix

A matrix captures the co-occurrence frequencies of two entities, as illustrated by a table pairing items like "raspberry" and "car" with attributes such as "red," "tasty," "fast," and "second-hand." The numerical values within the grid represent how often specific entity pairs appear together, providing a structured representation of these relationships for computational analysis.

Slide 36: Matrix

A matrix captures the co-occurrence frequencies of two entities, as demonstrated by a table cross-referencing items like "raspberry," "strawberry," "car," and "truck" against attributes such as "red," "tasty," "fast," and "second-hand." The numerical values within the grid represent how often each specific pair appears together, with non-zero entries indicating observed associations while zeros denote the absence of co-occurrence.

Slide 37: Matrix

A matrix captures the co-occurrence frequencies of two entities within a corpus, such as how often specific nouns appear with particular adjectives. For instance, "raspberry" and "strawberry" frequently co-occur with "red" and "tasty," while "car" and "truck" associate with "fast" and "second-hand" but not with "tasty." This structure quantifies these relationships by recording exact counts or zeros for each entity pair.

Slide 38: Matrix

A matrix captures the co-occurrence frequencies of two entities, as illustrated by a table where rows represent items like "raspberry," "strawberry," "car," and "truck," while columns represent attributes such as "red," "tasty," "fast," and "second-hand." The numerical values within the matrix quantify how often each specific entity-attribute pair appears together, providing a structured representation of these relationships for computational analysis.

Slide 39: Neural network: basics

A single neuron in a neural network functions identically to logistic regression, serving as the fundamental building block for these models. Like logistic regression, each neuron calculates the probability of an instance belonging to a specific class by weighting its input features with model parameters. This computation is mathematically expressed through matrix algebra, where inputs are multiplied by weight matrices and combined with bias factors to generate predictions.

Slide 40: Logistic regression

Logistic regression is applied to spam classification by using word frequencies as features, where each feature f_i is assigned a specific weight w_i. Words like "pharmacy" and "viagra" receive positive weights (0.4 and 1.2), indicating spam likelihood, while words such as "request" and "sending" have negative weights (-0.8 and -1.7). A bias term of 0.1 is also included in the model to adjust the classification threshold.

Slide 41: Logistic regression

Logistic regression is applied to spam classification by using word frequencies as features, where each word has an associated weight. For the email "Pharmacy online: viagra best offer!", the score is calculated by summing the weights of present words (pharmacy, viagra, best, offer) and the bias, while absent words contribute zero. This results in a final score of 2.1 for the given example.

Slide 42: Logistic regression

Binary logistic regression classifies emails as spam by calculating a linear score from word frequencies multiplied by specific weights, plus a bias term representing the prior probability of spam. This raw score, which ranges from negative to positive infinity, is transformed via the logistic function into a probability between 0 and 1. For instance, an email containing "pharmacy" and "viagra" yields a high spam probability (0.89), while one with "request" and "sending" results in a low probability (0.12).

Slide 43: Logistic regression

Logistic regression addresses the difficulty of interpreting linear scores, which range from negative to positive infinity, by transforming them into probabilities using a non-linear logistic function. This transformation applies the formula $1 / (1 + e^{-x})$ to map scores onto a 0-1 scale. For instance, a score of -2.1 yields a probability of 0.89, while a score of 2.0 results in a probability of 0.12.

Slide 44: Neuron = logistic regression

A single neuron performs the exact computation of logistic regression: inputs are multiplied by weights, summed with a bias term, and passed through a non-linear activation function (the logistic function) to map values from negative infinity to positive infinity into the range of zero to one. In a neural network, multiple neurons in a layer apply this same computation with different weights to an input vector, generating an output vector that serves as the input for subsequent layers. These weight matrices and bias vectors constitute the network's parameters, which are…

Slide 45: Feed-forward

This slide defines a feed-forward multi-layer perceptron with two hidden layers, where an input vector of size 3 is transformed into an output vector of size 2 via intermediate hidden vectors of size 4. The architecture computes representations sequentially: the first layer applies weights (3×4) and bias to generate h1, which serves as input for the second layer using a 4×4 weight matrix to produce h2. Finally, a 4×2 weight matrix transforms h2 into the output y, completing the basic neural network structure.

Slide 46: Feed-forward

Slide 46 defines a feed-forward multi-layer perceptron with two hidden layers, where the output y is computed through sequential transformations: $h_1 = g(xW_1 + b_1)$ and $h_2 = g(h_1W_2 + b_2)$, followed by $y = h_2W_3$. The architecture utilizes weight matrices W_1, W_2, W_3 of sizes [3×4], [4×4], and [4×2] respectively, along with bias vectors b_1, b_2 of size 4. The function $g(\cdot)$ represents an elementwise non-linear activation applied at each hidden layer.

Slide 47: Feed-forward

Slide 47 defines a feed-forward architecture as a multi-layer perceptron with two hidden layers, mathematically expressed through sequential transformations involving weights (W), biases (b), and an activation function (g). The network parameters, denoted as θ, include W1, W2, W3, b1, and b2. The slide explicitly identifies the use of multiple hidden layers as the defining characteristic of deep learning.

Slide 48: Training

Training aims to predict correct labels by minimizing a loss function, $L(\hat{y}, y)$, which quantifies the mismatch between predicted output $\hat{y}$ and true output y. To achieve this, parameters θ are adjusted by computing the gradient of the loss function with respect to those parameters. This mathematical process guides the model in the direction that reduces the cost incurred from incorrect predictions, ultimately ensuring accurate label prediction on the training set.

Slide 49: Training

Training utilizes the categorical cross-entropy loss function to quantify the difference between the true class distribution and the predicted distribution generated by a softmax output. While detailed mathematical derivations are outside this course's scope, the core objective is to optimize network parameters stochastically based on this quantified error against gold standard data.

Slide 50: Training

Stochastic gradient descent minimizes total loss by computing gradients over small batches rather than the entire training set, avoiding computationally heavy full-corpus calculations. This approach updates parameters incrementally based on sampled examples instead of waiting for a complete pass. The learning rate hyperparameter controls step size, requiring a balance between large steps for speed and small steps to prevent overshooting the optimal destination.

Slide 51: Training

Training utilizes stochastic gradient descent, where mini-batch size balances estimation quality against convergence speed. The backpropagation algorithm efficiently calculates gradients for parameters like weights and embeddings by propagating error through a computational graph to minimize prediction mismatch. While understanding these mathematical computations is essential, practitioners rely on existing automatic tools rather than performing manual derivations.

Slide 52: Word embeddings

Word embeddings represent each word as a small, dense vector in a real-valued, low-dimensional space, typically with dimensions between 50 and 1000. These vectors form a matrix of size vocabulary by embedding dimension, where similar words possess similar vector representations. Unlike interpretable co-occurrence counts, the individual dimensions of these embeddings are not readily interpretable, yet they effectively capture semantic relationships that allow for computing word similarity.

Slide 53: Word embeddings

Word embeddings represent each word as a small, dense vector in a low-dimensional real-valued space, typically with dimensions between 50 and 1000. Unlike traditional statistical models where words like "apple" and "pear" are represented by perpendicular vectors with no similarity, embeddings ensure that semantically similar words have closer vector representations. This approach allows the model to capture meaning by positioning related words nearer to each other compared to unrelated terms like "car."

Slide 54: Word embeddings

Word embeddings represent each word i as a small, dense vector $v_i \in \mathbb{R}^d$, where the dimension d typically ranges from 50 to 1000. These vectors form a matrix of size $V \times d$, embedding words into a real-valued, low-dimensional space. A key property is that semantically similar words possess similar vector embeddings within this space.

Slide 55: Neural word embeddings

Neural word embeddings represent each word as a dense vector of parameters (e.g., 50-300 values) rather than a single discrete value, forming an embedding matrix for the entire vocabulary. These vectors are automatically learned by training a neural network to predict the next word in a sequence based on preceding context words. By optimizing these embeddings to minimize prediction error, the network captures semantic similarities between words, enabling richer meaning representations useful for downstream tasks like sentiment analysis.

Slide 56: Embeddings through language modeling

This slide introduces generating embeddings by training a neural network on the language modeling task: predicting the next word in a sequence based on the previous one. The architecture consists of one non-linear hidden layer and a softmax classification layer, with parameters initialized randomly and optimized to maximize the probability of the correct word. This approach allows the network to learn meaningful vector representations autonomously rather than relying on discrete word counts or pre-existing values.

Slide 57: Embeddings through language modeling

Embeddings through language modeling involve predicting the next word in a sequence based on the previous word. This approach utilizes a neural network architecture consisting of one non-linear hidden layer and one softmax layer for classification. The model parameters are selected to optimize the probability of correctly predicting the target word.

Slide 58: Embeddings through error detection

Embeddings through error detection involve taking a correct sentence and creating a corrupted counterpart by randomly altering words, such as changing "white" to "heavy." The network is trained to assign a higher score to the correct version and a lower score to the corrupted one. This optimization process allows the model to learn an embedding matrix that captures semantic similarity between words.

Slide 59: Embeddings through error detection

Embeddings through error detection involve taking a correct sentence and generating a corrupted counterpart. The network is then trained to assign a higher score to the correct version of each sentence compared to its corrupted version, thereby learning meaningful representations through this discrimination task.

Slide 60: Word2vec

Word2vec is a popular, lightweight neural network approach for generating word embeddings that uses fewer parameters and no hidden layers compared to heavier models. It relies on a softmax classifier and implements two distinct architectures: Continuous Bag of Words (CBOW), which predicts the current word from surrounding context, and Skip-gram, which predicts surrounding words based on the current context word.

Slide 61: CBOW

Continuous Bag of Words (CBOW) predicts a center word from the embeddings of its surrounding context words, commonly by combining them with a sum or average. Because it treats the context as a bag, word order within that context is not represented; Skip-gram reverses the prediction direction and predicts context words from the center word.

Slide 62: Skip-gram

The Skip-gram model reverses the previous approach by taking a target word as input to predict its surrounding context words, such as predicting context for "wine." Unlike models with hidden layers, this architecture lacks one, making parameter estimation easier and the approach more lightweight. Additionally, the parameters for each softmax classifier used in these predictions are shared across the model.

Slide 63: Negative sampling

Negative sampling addresses the high computational cost of full softmax classifiers by calculating scores only for the correct context and a selected number of incorrect contexts. The objective is to maximize the score for the correct context while simultaneously minimizing the scores for the wrong ones, thereby reducing the overall computation required during training.

Slide 64: Word similarity

Word similarity is determined by computing the angle and cosine between word vectors to identify the most similar neighbors for a target word. For instance, 'Belgium' retrieves countries like 'Netherlands', while 'Jesus' yields related terms such as 'Christ' and 'Resurrection'. Similarly, color words like 'red' associate with 'yellow' and 'pink', and art forms like 'painting' link to 'sculpture' and 'watercolor', demonstrating how embeddings capture semantic relationships.

Slide 65: Word similarity

Word similarity compares lexical items by their vector representations. In an embedding space, nearby vectors tend to correspond to words with similar usage, though the learned relation depends on the training data and measure.

Slide 66: Word similarity

This build continues the word-similarity example. Similarity is a geometric relation between vectors, commonly measured with cosine similarity rather than raw distance when vector magnitude is not the focus.

Slide 67: Word similarity

The illustrated comparison shows how a similarity measure ranks words relative to one another. A high score means the vectors point in similar directions; it does not guarantee the words are interchangeable in every context.

Slide 68: Word similarity

This animation adds another step to the word-similarity comparison. Keep the score tied to the displayed embedding space and measure; it is not a universal semantic truth.

Slide 69: Word similarity

The final build closes the similarity example and prepares for vector arithmetic. The next analogy section uses relations between vectors, which are more limited than broad human analogical reasoning.

Slide 70: Computing analogies

Computing analogies involve questions structured as "a is to b as c is to d," where a system receives three terms and must compute the fourth. Examples include syntactic analogies like "apple is to apples as car is to ?" and semantic ones such as "man is to woman as king is to ?." The objective is to determine if the system can successfully solve these analogy problems based on the provided inputs.

Slide 71: Computing analogies

Computing analogies relies on the principle that relational directions remain consistent within a high-dimensional vector space, expressed as $a - b \approx c - d$. To solve for an unknown term like "queen" in "man is to woman as king is to ?", one calculates the vector $c - a + b$ and identifies the vocabulary word with the highest cosine similarity to this result. This process involves iterating through all embeddings to find the vector that best matches the computed directional relationship.

Slide 72: Computing analogies

This page introduces computing analogies with word vectors. A common illustration compares vector offsets between word pairs, such as a relation between a country and its capital; the result depends on the learned space.

Slide 73: Computing analogies

This build develops the vector-analogy example. Vector arithmetic tests whether a relation represented by one pair is approximately preserved for another pair.

Slide 74: Computing analogies

The analogy example is an empirical property of the learned embeddings, not a symbolic rule. It can fail when data are sparse, meanings are ambiguous, or the relation is not represented consistently.

Slide 75: Computing analogies

Slide 75 demonstrates that embedding matrices automatically capture relational information, enabling networks to solve analogies like "France is to Paris as Italy is to Rome" by identifying consistent vector directions. While the model successfully handles country-capital pairs and comparative adjectives, it does not always produce optimal results, such as associating France with tapas instead of sushi. These examples from Mikolov et al. (2013) illustrate that rich 300-dimensional spaces encode semantic relationships, though this raises broader questions regarding network interpretability.

Slide 76: Prediction-based vs. count-based

Count-based methods extract co-occurrence frequencies from a corpus to create explicit matrices, while prediction-based approaches induce parameters by predicting the correct word to generate embeddings. Although initially viewed as distinct with prediction-based methods performing better, emerging evidence proves they are equivalent because both fundamentally rely on co-occurrence data. Applying dimensionality reduction to count-based matrices yields results similar to those from prediction-based embedding approaches.

Slide 77: Word embeddings in practice

Word embeddings are frequently used for pretraining because they are unsupervised, requiring only plain text to leverage large datasets with fast algorithms. This approach allows models to start from an informed position by initializing them with pretrained embeddings before fine-tuning for specific tasks.

Key points to remember

The second half of the lecture transitions to machine learning architectures, detailing logistic regression as a basis for classification tasks like spam detection, where word frequencies act as features weighted against a bias term and transformed via a logistic function into probabilities. This concept extends to neural networks, specifically Feed-Forward Multi-Layer Perceptrons (MLPs), where neurons perform logistic regression operations across multiple hidden layers to capture complex patterns through non-linear activation functions. Training these models involves minimizing a loss function, such as categorical cross-entropy, using Stochastic Gradient Descent (SGD) with backpropagation to efficiently update parameters via mini-batches. Finally, the lecture covers word embeddings, which represent words as dense vectors in a low-dimensional space where semantic similarity corresponds to vector proximity, learned either through language modeling tasks that predict next words or error detection tasks that distinguish correct sentences from corrupted versions.